```
# EXPERIMENT NO. 7
# Motion Estimation using Optical Flow Algorithms in Video Sequences
# Algorithms:
# 1. Lucas-Kanade Sparse Optical Flow
# 2. Farneback Dense Optical Flow
# STEP 1: INSTALL REQUIRED TOOLS AND IMPORT LIBRARIES
!apt-get -qq update
!apt-get -qq install -y ffmpeg
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
import math
import time
import subprocess
import base64
from google.colab import files
from IPython.display import display, HTML
print("=" * 75)
print("EXPERIMENT NO. 7")
print("Motion Estimation using Optical Flow Algorithms in Video Sequences")
print("=" * 75)
print()
print("OpenCV Version :", cv2.__version__)
print("NumPy Version  :", np.__version__)
print("FFmpeg         : Ready")
print()
# STEP 2: UPLOAD VIDEO FROM PC
print("=" * 75)
print("STEP 1 - SELECT INPUT VIDEO")
print("=" * 75)
print()
print("A file selection window will appear.")
```

```
print("Select a video from your PC.")
print()
print("Recommended:")
print("• MP4 format")
print("• 10-30 seconds")
print("• 480p or 720p")
print()
uploaded = files.upload()
if not uploaded:
    raise RuntimeError(
        "No video was selected. Please run the cell again."
    )
video_filename = list(uploaded.keys())[0]
print()
print("Video uploaded successfully!")
print("Selected Video :", video_filename)
valid_extensions = [
    ".mp4", ".avi", ".mov",
    ".mkv", ".webm", ".mpeg", ".mpg"
]
extension = os.path.splitext(
    video_filename
)[1].lower()
print("File Extension :", extension)
if extension not in valid_extensions:
    print("WARNING: This format may not be fully supported.")
    print("MP4 is recommended.")
print()
# STEP 3: OPEN VIDEO AND GET INFORMATION
cap = cv2.VideoCapture(video_filename)
if not cap.isOpened():
    raise RuntimeError(
        "ERROR: OpenCV could not open the uploaded video."
```

```
    )
fps = cap.get(cv2.CAP_PROP_FPS)
frame_width = int(
    cap.get(cv2.CAP_PROP_FRAME_WIDTH)
)
frame_height = int(
    cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
)
total_frames = int(
    cap.get(cv2.CAP_PROP_FRAME_COUNT)
)
if fps <= 0 or np.isnan(fps):
    fps = 30.0
duration = (
    total_frames / fps
    if fps > 0
    else 0
)
print("=" * 75)
print("VIDEO INFORMATION")
print("=" * 75)
print(f"File Name    : {video_filename}")
print(f"Width        : {frame_width} pixels")
print(f"Height       : {frame_height} pixels")
print(f"FPS          : {fps:.2f}")
print(f"Total Frames : {total_frames}")
print(f"Duration     : {duration:.2f} seconds")
print("=" * 75)
print()
# STEP 4: DISPLAY FIRST FRAME
cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
ret, first_frame = cap.read()
if not ret:
```

```
    raise RuntimeError(
        "Could not read the first frame."
    )
first_frame_rgb = cv2.cvtColor(
    first_frame,
    cv2.COLOR_BGR2RGB
)
plt.figure(figsize=(12, 6))
plt.imshow(first_frame_rgb)
plt.title(
    "First Frame of Input Video",
    fontsize=15
)
plt.axis("off")
plt.show()
# STEP 5: OPTICAL FLOW PARAMETERS
# ---------------------- Lucas-Kanade ----------------------
feature_params = dict(
    maxCorners=200,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)
lk_params = dict(
    winSize=(21, 21),
    maxLevel=3,
    criteria=(
        cv2.TERM_CRITERIA_EPS |
        cv2.TERM_CRITERIA_COUNT,
        30,
        0.01
    )
)
```

```
# ---------------------- Farneback ----------------------
farneback_params = dict(
    pyr_scale=0.5,
    levels=3,
    winsize=15,
    iterations=3,
    poly_n=5,
    poly_sigma=1.2,
    flags=0
)
print("=" * 75)
print("OPTICAL FLOW PARAMETERS")
print("=" * 75)
print("Lucas-Kanade parameters configured.")
print("Farneback parameters configured.")
print()
# STEP 6: OUTPUT FILE NAMES
# Intermediate files
sparse_temp = "Experiment_07_Lucas_Kanade_TEMP.avi"
dense_temp = "Experiment_07_Farneback_TEMP.avi"
comparison_temp = "Experiment_07_Comparison_TEMP.avi"
# Final files
sparse_output = "Experiment_07_Lucas_Kanade.mp4"
dense_output = "Experiment_07_Farneback.mp4"
comparison_output = "Experiment_07_Optical_Flow_Comparison.mp4"
# STEP 7: CREATE INTERMEDIATE VIDEO WRITERS
# MJPG is used as an intermediate format.
# FFmpeg will later convert it to browser-friendly H.264 MP4.
fourcc = cv2.VideoWriter_fourcc(
    *"MJPG"
)
sparse_writer = cv2.VideoWriter(
    sparse_temp,
```

```
    fourcc,
    fps,
    (
        frame_width,
        frame_height
    )
)
dense_writer = cv2.VideoWriter(
    dense_temp,
    fourcc,
    fps,
    (
        frame_width,
        frame_height
    )
)
comparison_writer = cv2.VideoWriter(
    comparison_temp,
    fourcc,
    fps,
    (
        frame_width * 2,
        frame_height
    )
)
if not sparse_writer.isOpened():
    raise RuntimeError(
        "Lucas-Kanade video writer failed."
    )
if not dense_writer.isOpened():
    raise RuntimeError(
        "Farneback video writer failed."
    )
```

```
if not comparison_writer.isOpened():
    raise RuntimeError(
        "Comparison video writer failed."
    )
print("=" * 75)
print("VIDEO WRITERS CREATED")
print("=" * 75)
print()
# STEP 8: INITIALIZE FIRST FRAME
cap.set(
    cv2.CAP_PROP_POS_FRAMES,
    0
)
ret, old_frame = cap.read()
if not ret:
    raise RuntimeError(
        "Could not read the first video frame."
    )
old_gray = cv2.cvtColor(
    old_frame,
    cv2.COLOR_BGR2GRAY
)
# Detect initial feature points
p0 = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    **feature_params
)
# Trajectory drawing mask
trajectory_mask = np.zeros_like(
    old_frame
)
print(
```

```
    "Initial feature points detected:",
    0 if p0 is None else len(p0)
)
print()
# STEP 9: CREATE STORAGE FOR STATISTICS
frame_numbers = []
lk_average_motion = []
lk_max_motion = []
lk_tracked_points = []
fb_average_magnitude = []
fb_max_magnitude = []
processing_times = []
# Sample frames
sample_frames = []
sample_lk_frames = []
sample_fb_frames = []
sample_positions = set(
    np.linspace(
        0,
        max(total_frames - 2, 1),
        5,
        dtype=int
    ).tolist()
)
# STEP 10: MAIN OPTICAL FLOW PROCESSING
print("=" * 75)
print("OPTICAL FLOW PROCESSING STARTED")
print("=" * 75)
print()
frame_index = 0
start_time = time.time()
while True:
    ret, frame = cap.read()
```

```
if not ret:
    break
processing_start = time.time()
# CONVERT CURRENT FRAME TO GRAYSCALE
frame_gray = cv2.cvtColor(
    frame,
    cv2.COLOR_BGR2GRAY
)
# LUCAS-KANADE SPARSE OPTICAL FLOW
lk_frame = frame.copy()
average_lk_motion = 0.0
maximum_lk_motion = 0.0
tracked_points_count = 0
# Re-detect feature points when necessary
if (
    p0 is None
    or len(p0) < 10
):
    p0 = cv2.goodFeaturesToTrack(
        old_gray,
        mask=None,
        **feature_params
    )
    trajectory_mask = np.zeros_like(
        frame
    )
if p0 is not None:
    # Calculate Lucas-Kanade optical flow
    p1, status, error = cv2.calcOpticalFlowPyrLK(
        old_gray,
        frame_gray,
        p0,
        None,
```

```
    **lk_params
)
if (
    p1 is not None
    and status is not None
):
    good_new = p1[
        status.ravel() == 1
    ]
    good_old = p0[
        status.ravel() == 1
    ]
    motion_values = []
    # DRAW MOTION VECTORS AND TRAJECTORIES
    for new, old in zip(
        good_new,
        good_old
    ):
        new_x, new_y = new.ravel()
        old_x, old_y = old.ravel()
        dx = (
            new_x -
            old_x
        )
        dy = (
            new_y -
            old_y
        )
        magnitude_value = math.sqrt(
            dx * dx +
            dy * dy
        )
        motion_values.append(
```

```
    magnitude_value
)
# Draw trajectory
trajectory_mask = cv2.line(
    trajectory_mask,
    (
        int(new_x),
        int(new_y)
    ),
    (
        int(old_x),
        int(old_y)
    ),
    (255, 255, 255),
    2
)
# Draw arrow
lk_frame = cv2.arrowedLine(
    lk_frame,
    (
        int(old_x),
        int(old_y)
    ),
    (
        int(new_x),
        int(new_y)
    ),
    (0, 255, 0),
    2,
    tipLength=0.3
)
# Draw feature point
lk_frame = cv2.circle(
```

```
            lk_frame,
            (
                int(new_x),
                int(new_y)
            ),
            4,
            (0, 0, 255),
            -1
        )
    # MOTION STATISTICS
    if len(motion_values) > 0:
        average_lk_motion = float(
            np.mean(
                motion_values
            )
        )
        maximum_lk_motion = float(
            np.max(
                motion_values
            )
        )
    tracked_points_count = len(
        good_new
    )
    # Add trajectories
    lk_frame = cv2.add(
        lk_frame,
        trajectory_mask
    )
    # Update feature points
    p0 = good_new.reshape(
        -1,
        1,
```

```
            2
        )
    else:
        p0 = cv2.goodFeaturesToTrack(
            frame_gray,
            mask=None,
            **feature_params
        )
        trajectory_mask = np.zeros_like(
            frame
        )
# FARNEBACK DENSE OPTICAL FLOW
flow = cv2.calcOpticalFlowFarneback(
    old_gray,
    frame_gray,
    None,
    **farneback_params
)
# Convert Cartesian flow to magnitude and angle
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)
# HSV VISUALIZATION
hsv = np.zeros_like(
    frame
)
# Saturation
hsv[..., 1] = 255
# Direction → Hue
hsv[..., 0] = (
    angle *
    180 /
```

```
    np.pi /
    2
).astype(
    np.uint8
)
# Magnitude → Brightness
hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(
    np.uint8
)
# Convert HSV → BGR
farneback_frame = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2BGR
)
# Dense motion statistics
average_fb_magnitude = float(
    np.mean(
        magnitude
    )
)
maximum_fb_magnitude = float(
    np.max(
        magnitude
    )
)
# ADD TEXT TO LUCAS-KANADE RESULT
cv2.putText(
```

```
    lk_frame,
    "Lucas-Kanade Sparse Optical Flow",
    (20, 35),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.75,
    (0, 255, 255),
    2
)
cv2.putText(
    lk_frame,
    f"Tracked Points: {tracked_points_count}",
    (20, 70),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (255, 255, 255),
    2
)
cv2.putText(
    lk_frame,
    f"Avg Motion: {average_lk_motion:.2f} px",
    (20, 100),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (255, 255, 255),
    2
)
# ADD TEXT TO FARNEBACK RESULT
cv2.putText(
    farneback_frame,
    "Farneback Dense Optical Flow",
    (20, 35),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.75,
```

```
        (255, 255, 255),
        2
    )
    cv2.putText(
        farneback_frame,
        f"Avg Magnitude: {average_fb_magnitude:.2f}",
        (20, 70),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (255, 255, 255),
        2
    )
    cv2.putText(
        farneback_frame,
        f"Max Magnitude: {maximum_fb_magnitude:.2f}",
        (20, 100),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (255, 255, 255),
        2
    )
# SIDE-BY-SIDE COMPARISON
    comparison_frame = np.hstack(
        (
            lk_frame,
            farneback_frame
        )
    )
    # Divider
    cv2.line(
        comparison_frame,
        (
            frame_width,
```

```
        0
    ),
    (
        frame_width,
        frame_height
    ),
    (255, 255, 255),
    3
)
# Sparse label
cv2.putText(
    comparison_frame,
    "SPARSE",
    (
        20,
        frame_height - 20
    ),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (255, 255, 255),
    2
)
# Dense label
cv2.putText(
    comparison_frame,
    "DENSE",
    (
        frame_width + 20,
        frame_height - 20
    ),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (255, 255, 255),
```

```
        2
    )
    # WRITE OUTPUT VIDEOS
    sparse_writer.write(
        lk_frame
    )
    dense_writer.write(
        farneback_frame
    )
    comparison_writer.write(
        comparison_frame

    # SAVE STATISTICS
frame_numbers.append(
        frame_index
    )
    lk_average_motion.append(
        average_lk_motion
    )
    lk_max_motion.append(
        maximum_lk_motion
    )
    lk_tracked_points.append(
        tracked_points_count
    )
    fb_average_magnitude.append(
        average_fb_magnitude
    )
    fb_max_magnitude.append(
        maximum_fb_magnitude
    )
    processing_times.append(
        time.time() -
```

```
    processing_start
)
# SAVE SAMPLE FRAMES
if frame_index in sample_positions:
    sample_frames.append(
        cv2.cvtColor(
            frame,
            cv2.COLOR_BGR2RGB
        )
    )
    sample_lk_frames.append(
        cv2.cvtColor(
            lk_frame,
            cv2.COLOR_BGR2RGB
        )
    )
    sample_fb_frames.append(
        cv2.cvtColor(
            farneback_frame,
            cv2.COLOR_BGR2RGB
        )
    )
# UPDATE PREVIOUS FRAME
old_gray = frame_gray.copy()
frame_index += 1
# PROGRESS
if frame_index % 30 == 0:
    if total_frames > 0:
        progress = (
            frame_index /
            total_frames
        ) * 100
        print(
```

```
                f"Processed: "
                f"{frame_index}/{total_frames} "
                f"({progress:.1f}%)"
            )
# RELEASE VIDEO RESOURCES
cap.release()
sparse_writer.release()
dense_writer.release()
comparison_writer.release()
total_processing_time = (
    time.time() -
    start_time
)
print()
print("=" * 75)
print("OPTICAL FLOW PROCESSING COMPLETED")
print("=" * 75)
print(
    "Frames Processed :",
    frame_index
)
print(
    "Processing Time  :",
    f"{total_processing_time:.2f} seconds"
)
print()
# STEP 11: VERIFY INTERMEDIATE FILES
print("=" * 75)
print("INTERMEDIATE FILE VERIFICATION")
print("=" * 75)
for file_name in [
    sparse_temp,
    dense_temp,
```

```
    comparison_temp
]:
    if os.path.exists(
        file_name
    ):
        size_mb = (
            os.path.getsize(
                file_name
            )
            /
            (1024 * 1024)
        )
        print(
            f"✓ {file_name} "
            f"({size_mb:.2f} MB)"
        )
    else:
        print(
            f"✗ {file_name} NOT FOUND"
        )
print()
# STEP 12: CONVERT AVI → H.264 MP4
def convert_to_h264(
    input_file,
    output_file
):
    print(
        f"Converting {input_file} ..."
    )
    command = [
        "ffmpeg",
        "-y",
        "-i",
```

```
        input_file,
        "-c:v",
        "libx264",
        "-preset",
        "fast",
        "-crf",
        "23",
        "-pix_fmt",
        "yuv420p",
        "-movflags",
        "+faststart",
        output_file
    ]
    result = subprocess.run(
        command,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True
    )
    if result.returncode != 0:
        print(
            result.stderr
        )
        raise RuntimeError(
            f"FFmpeg conversion failed "
            f"for {input_file}"
        )
    print(
        f"✓ Created: {output_file}"
    )
print("=" * 75)
print("CONVERTING OUTPUT VIDEOS")
print("=" * 75)
```

```
convert_to_h264(
    sparse_temp,
    sparse_output
)
convert_to_h264(
    dense_temp,
    dense_output
)
convert_to_h264(
    comparison_temp,
    comparison_output
)
print()
print("All final MP4 files created.")
print()
# STEP 13: VERIFY FINAL MP4 FILES
print("=" * 75)
print("FINAL MP4 FILE VERIFICATION")
print("=" * 75)
final_files = [
    sparse_output,
    dense_output,
    comparison_output
]
for file_name in final_files:
    if os.path.exists(
        file_name
    ):
        size_mb = (
            os.path.getsize(
                file_name
            )
            /
```

```
            (1024 * 1024)
        )
        print(
            f"✓ {file_name}"
        )
        print(
            f"  Size: {size_mb:.2f} MB"
        )
    else:
        print(
            f"✗ {file_name} NOT FOUND"
        )
print()
# STEP 14: FUNCTION TO DISPLAY MP4 VIDEO
def display_video_html(
    filename,
    width=800
):
    if not os.path.exists(
        filename
    ):
        print(
            "Video not found:",
            filename
        )
        return
    with open(
        filename,
        "rb"
    ) as video_file:
        video_data = base64.b64encode(
            video_file.read()
        ).decode(
```

```
            "utf-8"
        )
    html = f"""
    <video
        width="{width}"
        controls
        preload="metadata"
    >
        <source
            src="data:video/mp4; base64,{video_data}"
            type="video/mp4"
        >
        Your browser does not support
        HTML5 video.
    </video>
    """
    display(
        HTML(html)
    )
# STEP 15: DISPLAY LUCAS-KANADE VIDEO
print("=" * 75)
print("LUCAS-KANADE SPARSE OPTICAL FLOW OUTPUT")
print("=" * 75)
display_video_html(
    sparse_output,
    width=800
)
print()
# STEP 16: DISPLAY FARNEBACK VIDEO
print("=" * 75)
print("FARNEBACK DENSE OPTICAL FLOW OUTPUT")
print("=" * 75)
display_video_html(
```

```
    dense_output,
    width=800
)
print()
# STEP 17: DISPLAY SIDE-BY-SIDE VIDEO
print("=" * 75)
print("SIDE-BY-SIDE COMPARISON OUTPUT")
print("=" * 75)
display_video_html(
    comparison_output,
    width=1000
)
print()
# STEP 18: DISPLAY SAMPLE ORIGINAL FRAMES
if len(sample_frames) > 0:
    print("=" * 75)
    print("SAMPLE ORIGINAL FRAMES")
    print("=" * 75)
    n = len(
        sample_frames
    )
    columns = 3
    rows = math.ceil(
        n / columns
    )
    plt.figure(
        figsize=(16, 8)
    )
    for i, image in enumerate(
        sample_frames
    ):
        plt.subplot(
            rows,
```

```
            columns,
            i + 1
        )
        plt.imshow(
            image
        )
        plt.title(
            f"Original Frame {i + 1}"
        )
        plt.axis(
            "off"
        )
    plt.tight_layout()
    plt.show()
# STEP 19: DISPLAY LUCAS-KANADE SAMPLE RESULTS
if len(sample_lk_frames) > 0:
    print("=" * 75)
    print("LUCAS-KANADE SAMPLE RESULTS")
    print("=" * 75)
    n = len(
        sample_lk_frames
    )
    columns = 3
    rows = math.ceil(
        n / columns
    )
    plt.figure(
        figsize=(16, 8)
    )
    for i, image in enumerate(
        sample_lk_frames
    ):
        plt.subplot(
```

```
            rows,
            columns,
            i + 1
        )
        plt.imshow(
            image
        )
        plt.title(
            f"Lucas-Kanade Frame {i + 1}"
        )
        plt.axis(
            "off"
        )
    plt.tight_layout()
    plt.show()
# STEP 20: DISPLAY FARNEBACK SAMPLE RESULTS
if len(sample_fb_frames) > 0:
    print("=" * 75)
    print("FARNEBACK SAMPLE RESULTS")
    print("=" * 75)
    n = len(
        sample_fb_frames
    )
    columns = 3
    rows = math.ceil(
        n / columns
    )
    plt.figure(
        figsize=(16, 8)
    )
    for i, image in enumerate(
        sample_fb_frames
    ):
```

```
        plt.subplot(
            rows,
            columns,
            i + 1
        )
        plt.imshow(
            image
        )
        plt.title(
            f"Farneback Frame {i + 1}"
        )
        plt.axis(
            "off"
        )
    plt.tight_layout()
    plt.show()
# STEP 21: LUCAS-KANADE MOTION GRAPH
if len(frame_numbers) > 0:
    plt.figure(
        figsize=(12, 5)
    )
    plt.plot(
        frame_numbers,
        lk_average_motion,
        label="Average Motion"
    )
    plt.plot(
        frame_numbers,
        lk_max_motion,
        label="Maximum Motion"
    )
    plt.xlabel(
        "Frame Number"
```

```
    )
    plt.ylabel(
        "Motion / Displacement (pixels)"
    )
    plt.title(
        "Lucas-Kanade Sparse Optical Flow Motion Analysis"
    )
    plt.legend()
    plt.grid(
        True
    )
    plt.tight_layout()
    plt.show()
# STEP 22: FARNEBACK MOTION GRAPH
if len(frame_numbers) > 0:
    plt.figure(
        figsize=(12, 5)
    )
    plt.plot(
        frame_numbers,
        fb_average_magnitude,
        label="Average Magnitude"
    )
    plt.plot(
        frame_numbers,
        fb_max_magnitude,
        label="Maximum Magnitude"
    )
    plt.xlabel(
        "Frame Number"
    )
    plt.ylabel(
        "Optical Flow Magnitude"
```

```
    )
    plt.title(
        "Farneback Dense Optical Flow Motion Analysis"
    )
    plt.legend()
    plt.grid(
        True
    )
    plt.tight_layout()
    plt.show()
# STEP 23: TRACKED FEATURE POINT GRAPH
if len(frame_numbers) > 0:
    plt.figure(
        figsize=(12, 5)
    )
    plt.plot(
        frame_numbers,
        lk_tracked_points,
        label="Tracked Feature Points"
    )
    plt.xlabel(
        "Frame Number"
    )
    plt.ylabel(
        "Number of Points"
    )
    plt.title(
        "Lucas-Kanade Tracked Feature Points"
    )
    plt.legend()
    plt.grid(
        True
    )
```

```
    plt.tight_layout()
    plt.show()
# STEP 24: PERFORMANCE ANALYSIS
if len(processing_times) > 0:
    average_frame_time = np.mean(
        processing_times
    )
    processing_fps = (
        1 /
        average_frame_time
        if average_frame_time > 0
        else 0
    )
    mean_lk_motion = np.mean(
        lk_average_motion
    )
    max_lk_motion = np.max(
        lk_max_motion
    )
    mean_fb_motion = np.mean(
        fb_average_magnitude
    )
    max_fb_motion = np.max(
        fb_max_magnitude
    )
    average_tracked_points = np.mean(
        lk_tracked_points
    )
    print("=" * 75)
    print("EXPERIMENT 07 - PERFORMANCE ANALYSIS")
    print("=" * 75)
    print(
        f"Frames Processed        : "
```

```
    f"{frame_index}"
)
print(
    f"Total Processing Time   : "
    f"{total_processing_time:.2f} seconds"
)
print(
    f"Average Time / Frame    : "
    f"{average_frame_time:.4f} seconds"
)
print(
    f"Approx. Processing FPS  : "
    f"{processing_fps:.2f}"
)
print()
print(
    f"Average LK Motion       : "
    f"{mean_lk_motion:.4f} px"
)
print(
    f"Maximum LK Motion       : "
    f"{max_lk_motion:.4f} px"
)
print(
    f"Average Tracked Points  : "
    f"{average_tracked_points:.2f}"
)
print()
print(
    f"Average Farneback Flow  : "
    f"{mean_fb_motion:.4f}"
)
print(
```

```
        f"Maximum Farneback Flow  : "
        f"{max_fb_motion:.4f}"
    )
    print("=" * 75)
    print()
# STEP 25: ALGORITHM COMPARISON
print("=" * 75)
print("LUCAS-KANADE VS FARNEBACK")
print("=" * 75)
comparison_table = [
    (
        "Type",
        "Sparse",
        "Dense"
    ),
    (
        "Motion Estimation",
        "Selected feature points",
        "Almost every pixel"
    ),
    (
        "Algorithm",
        "Lucas-Kanade",
        "Farneback"
    ),
    (
        "Output",
        "Motion vectors / trajectories",
        "Dense motion field"
    ),
    (
        "Feature Selection",
        "Required",
```

```
        "Not explicitly required"
    ),
    (
        "Computational Cost",
        "Generally lower",
        "Generally higher"
    ),
    (
        "Object Tracking",
        "Suitable",
        "Possible but heavier"
    ),
    (
        "Scene Motion Analysis",
        "Feature based",
        "Full image based"
    ),
    (
        "Visualization",
        "Arrows / trajectories",
        "Color-coded flow"
    )
]
print(
    f"{'Parameter':<28}"
    f"| {'Lucas-Kanade':<32}"
    f"| {'Farneback'}"
)
print("-" * 95)
for row in comparison_table:
    print(
        f"{row[0]:<28}"
        f"| {row[1]:<32}"
```

```
        f"| {row[2]}"
    )
print("=" * 75)
print()
# STEP 26: FINAL EXPERIMENT RESULT
print("=" * 75)
print("EXPERIMENT 07 - FINAL RESULT")
print("=" * 75)
print()
print(
    "Optical Flow was successfully implemented "
    "using Lucas-Kanade and Farneback algorithms."
)
print()
print(
    "Lucas-Kanade estimated motion of selected "
    "feature points between consecutive frames."
)
print()
print(
    "Farneback estimated dense motion across "
    "the image and represented it using colors."
)
print()
print(
    "Motion vectors, trajectories, dense flow "
    "visualization and motion graphs were generated."
)
print()
print(
    "The two algorithms were compared based on "
    "their motion representation and processing behavior."
)
```

```
print()
print(
    "RESULT: EXPERIMENT COMPLETED SUCCESSFULLY"
)
print()
# STEP 27: DOWNLOAD BUTTONS
download_html = f"""
<div style="
    font-family: Arial;
    padding: 20px;
    border: 2px solid #444;
    border-radius: 10px;
    max-width: 750px;
">
<h2>Experiment 07 - Output Videos</h2>
<p>
Download the generated optical-flow videos:
</p>
<a href="/files/{sparse_output}" download>
<button style="
    font-size:16px;
    padding:12px 22px;
    margin:8px;
    cursor:pointer;
">
Download Lucas-Kanade Video
</button>
</a>
<br>
<a href="/files/{dense_output}" download>
<button style="
    font-size:16px;
    padding:12px 22px;
```

```
    margin:8px;
    cursor:pointer;
">
Download Farneback Video
</button>
</a>
<br>
<a href="/files/{comparison_output}" download>
<button style="
    font-size:16px;
    padding:12px 22px;
    margin:8px;
    cursor:pointer;
">
Download Comparison Video
</button>
</a>
</div>
"""
display(
    HTML(
        download_html
    )
)
# STEP 28: FINAL FILE LIST
print()
print("=" * 75)
print("FINAL OUTPUT FILES")
print("=" * 75)
for file_name in final_files:
    if os.path.exists(
        file_name
    ):
```

```
        size_mb = (
            os.path.getsize(
                file_name
            )
            /
            (1024 * 1024)
        )
        print(
            f"✓ {file_name}"
        )
        print(
            f"  Size: {size_mb:.2f} MB"
        )
print()
print("=" * 75)
print("EXPERIMENT 07 COMPLETED")
print("=" * 75)
print()
print(
    "You can now play the videos above "
    "and download them to your PC."
)
print()
# END OF EXPERIMENT 07
```

OUTPUT –

EXPERIMENT NO. 7

```
Motion Estimation using Optical Flow Algorithms in Video Sequences
OpenCV Version : 5.0.0
NumPy Version  : 2.1.3
FFmpeg         : Ready
```

STEP 1 - SELECT INPUT VIDEO

A file selection window will appear. Select a video from your PC.

Recommended:

• MP4 format

• 10-30 seconds

• 480p or 720p

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving SAMPLE VIDEO FOR TESTING COMPUTER VISION.mp4 to SAMPLE VIDEO FOR TESTING COMPUTER VISION (1).mp4

Video uploaded successfully!

Selected Video : SAMPLE VIDEO FOR TESTING COMPUTER VISION (1).mp4

File Extension : .mp4

VIDEO INFORMATION

File Name : SAMPLE VIDEO FOR TESTING COMPUTER VISION (1).mp4

Width : 2560 pixels

Height : 1440 pixels

FPS : 29.97

Total Frames : 823

Duration : 27.46 seconds



```
OPTICAL FLOW PARAMETERS
Lucas-Kanade parameters configured.
Farneback parameters configured.
VIDEO WRITERS CREATED
Initial feature points detected: 200
OPTICAL FLOW PROCESSING STARTED
Processed: 30/823 (3.6%)
Processed: 60/823 (7.3%)
Processed: 90/823 (10.9%)
Processed: 120/823 (14.6%)
Processed: 150/823 (18.2%)
Processed: 180/823 (21.9%)
Processed: 210/823 (25.5%)
Processed: 240/823 (29.2%)
Processed: 270/823 (32.8%)
Processed: 300/823 (36.5%)
Processed: 330/823 (40.1%)
Processed: 360/823 (43.7%)
Processed: 390/823 (47.4%)
Processed: 420/823 (51.0%)
Processed: 450/823 (54.7%)
Processed: 480/823 (58.3%)
Processed: 510/823 (62.0%)
Processed: 540/823 (65.6%)
Processed: 570/823 (69.3%)
Processed: 600/823 (72.9%)
Processed: 630/823 (76.5%)
Processed: 660/823 (80.2%)
Processed: 690/823 (83.8%)
Processed: 720/823 (87.5%)
Processed: 750/823 (91.1%)
Processed: 780/823 (94.8%)
Processed: 810/823 (98.4%)
```

```
OPTICAL FLOW PROCESSING COMPLETED
Frames Processed : 822
Processing Time  : 922.74 seconds
INTERMEDIATE FILE VERIFICATION
✓ Experiment_07_Lucas_Kanade_TEMP.avi (594.56 MB)
✓ Experiment_07_Farneback_TEMP.avi (136.38 MB)
✓ Experiment_07_Comparison_TEMP.avi (745.58 MB)
CONVERTING OUTPUT VIDEOS
Converting Experiment_07_Lucas_Kanade_TEMP.avi ...
✓ Created: Experiment_07_Lucas_Kanade.mp4
Converting Experiment_07_Farneback_TEMP.avi ...
✓ Created: Experiment_07_Farneback.mp4
Converting Experiment_07_Comparison_TEMP.avi ...
✓ Created: Experiment_07_Optical_Flow_Comparison.mp4
All final MP4 files created.
FINAL MP4 FILE VERIFICATION
✓ Experiment_07_Lucas_Kanade.mp4
  Size: 49.17 MB
✓ Experiment_07_Farneback.mp4
  Size: 58.95 MB
✓ Experiment_07_Optical_Flow_Comparison.mp4
  Size: 109.85 MB
LUCAS-KANADE SPARSE OPTICAL FLOW OUTPUT
Buffered data was truncated after reaching the output
```